

# Quiz — Padrões GRASP

## General Responsibility Assignment Software Patterns

10 questões | Nível avançado | Dificuldade: ● fácil ●● médio ●●● difícil

### Questão 1 Expert (Information Expert) Dificuldade: ●●○

```
class Fatura {
    private List<LinhaFatura> linhas;
    private double desconto;

    public double calcularTotal() {
        double sub = linhas.stream()
            .mapToDouble(l -> l.getPreco() * l.getQtd())
            .sum();
        return sub * (1 - desconto);
    }
}
```

Ao mover `calcularTotal()` para uma classe `FaturaService` externa, qual violação de padrão GRASP é introduzida primariamente?

- A) Viola o Expert porque `FaturaService` não possui as informações necessárias para o cálculo.
- B) Viola o Creator porque `FaturaService` passa a criar objetos `LinhaFatura`.
- C) Viola o Controller porque a lógica de coordenação sai do domínio.
- D) Viola o Low Coupling porque `Fatura` precisaria expor todos os seus campos publicamente.
- E) Não viola nenhum padrão; `FaturaService` é uma solução arquitetural válida para qualquer situação.

### Questão 2 Creator Dificuldade: ●●○

```
class OrdemDeCompra {
    private List<ItemOrdem> itens;

    public void incluirItem(Produto produto, int qtd) {
        itens.add(new ItemOrdem(produto, qtd));
    }
}
```

Um colega sugere mover a criação de `ItemOrdem` para uma classe fábrica `ItemOrdemFactory`. Qual afirmativa melhor avalia essa proposta sob a ótica do GRASP Creator?

- A) A proposta é sempre preferível porque fábricas são um padrão GoF superior ao Creator.
- B) A proposta reduz o acoplamento entre `OrdemDeCompra` e `ItemOrdem`, sendo válida quando a lógica de criação for complexa ou variável.
- C) A proposta viola o Creator porque `OrdemDeCompra` agrega `ItemOrdem` e, portanto, jamais deve delegar a criação.

D) A proposta é inválida porque ItemOrdemFactory teria alta coesão e isso conflita com Creator.

E) A proposta só é válida se ItemOrdemFactory também gerenciar a lista de itens da OrdemDeCompra.

### Questão 3 High Cohesion Dificuldade: ●●●

```
class Usuario {
    private String nome;
    private String email;

    public boolean autenticar(String senha) { /* ... */ }
    public void enviarEmailBoas_vindas() { /* ... */ }
    public double calcularDescontoFidelidade() { /* ... */ }
    public String gerarRelatorioAtividades() { /* ... */ }
}
```

Qual par de métodos, se extraído para classes separadas, geraria o maior ganho de coesão na classe Usuario?

A) autenticar e enviarEmailBoas\_vindas, pois ambos lidam com o ciclo de vida do usuário.

B) calcularDescontoFidelidade e gerarRelatorioAtividades, pois nenhum dos dois depende do estado de Usuario.

C) enviarEmailBoas\_vindas e calcularDescontoFidelidade, pois são responsabilidades de domínios distintos (notificação e precificação).

D) autenticar e calcularDescontoFidelidade, pois segurança e precificação não pertencem à entidade usuário.

E) Nenhum par deve ser extraído; Usuario é uma entidade de domínio e pode acumular todas as responsabilidades.

### Questão 4 Low Coupling Dificuldade: ●●●

```
class Relatorio {
    public String gerar(Pedido pedido) {
        double total = pedido.calcularTotal();
        String cliente = pedido.getCliente().getNome();
        String status = pedido.getStatus().getDescricao();
        return cliente + " | " + total + " | " + status;
    }
}
```

O código acessa pedido.getCliente().getNome() e pedido.getStatus().getDescricao(). Qual problema de acoplamento isso representa e como o GRASP o nomeia?

A) Viola Low Coupling porque Relatorio depende de três classes (Pedido, Cliente, Status) em vez de uma.

B) Viola o Expert porque Relatorio não deveria ter acesso à estrutura interna de Pedido.

C) Viola o Controller porque Relatorio está coordenando a extração de dados de múltiplas classes.

D) Não viola nenhum padrão porque Relatorio apenas lê dados sem modificar estado.

E) Viola o Creator porque Relatorio instancia objetos implicitamente ao percorrer a cadeia.

### Questão 5 Controller Dificuldade: ●●●

```
class PedidoController {
    private EstoqueService estoque;
    private PagamentoService pagamento;
    private NotificacaoService notificacao;

    public ResultadoDTO finalizarPedido(PedidoDTO dto) {
        boolean reservado = estoque.reservar(dto.getItems());
        if (!reservado) return ResultadoDTO.falha("Sem estoque");
        boolean pago = pagamento.cobrar(dto.getValor());
        if (!pago) { estoque.liberar(dto.getItems()); return
ResultadoDTO.falha("Pagamento"); }
        notificacao.enviar(dto.getEmail());
        return ResultadoDTO.sucesso();
    }
}
```

Esse Controller coordena estoque, pagamento e notificação. Considerando o padrão GRASP Controller, qual crítica é mais precisa?

- A) O design está correto: Controller deve orquestrar todos os serviços do caso de uso.
- B) O Controller tem coesão baixa pois acumula lógica de negócio (compensação de estoque) que deveria estar num objeto de domínio ou serviço de aplicação.
- C) O Controller viola o Creator porque instancia ResultadoDTO diretamente.
- D) O Controller viola o Low Coupling porque depende de três serviços simultaneamente.
- E) O design está incorreto porque Controllers nunca devem chamar mais de um serviço por método.

### Questão 6 Polymorphism Dificuldade: ●●○

```
class Notificador {
    public void enviar(String tipo, String mensagem) {
        if (tipo.equals("email")) {
            // envia email
        } else if (tipo.equals("sms")) {
            // envia sms
        } else if (tipo.equals("push")) {
            // envia push
        }
    }
}
```

O padrão GRASP Polimorfismo sugere que esse if-else seja substituído por quê e por qual motivo?

- A) Por um switch-case, que é mais eficiente que if-else para múltiplas condições.
- B) Por uma hierarquia de classes (EmailNotificador, SmsNotificador, PushNotificador) com um método enviar() polimórfico, eliminando condicionais sobre tipo.
- C) Por um repositório que armazena os tipos e suas implementações em banco de dados.

- D) Por injeção de dependência, tornando o tipo uma configuração em vez de um parâmetro.
- E) Por um Controller que decide qual notificador chamar com base no tipo recebido.

### Questão 7 Pure Fabrication Dificuldade: ●●●

```
class Pedido {  
    public void salvar() {  
        // abre conexão JDBC  
        // monta INSERT SQL  
        // executa  
        // fecha conexão  
    }  
}
```

Mover a lógica de persistência para uma classe PedidoRepository é um exemplo de qual padrão GRASP e por quê?

- A) Creator, porque Repository passa a ser responsável por criar objetos Pedido ao carregá-los do banco.
- B) Pure Fabrication, porque Repository não representa um conceito do domínio do problema — é uma classe inventada para dar suporte à infraestrutura, mantendo Pedido coeso.
- C) Controller, porque Repository coordena as operações de leitura e escrita do sistema.
- D) Low Coupling, porque a separação elimina a dependência de Pedido com o banco de dados.
- E) High Cohesion, porque Repository concentra todas as responsabilidades de persistência.

### Questão 8 Indirection Dificuldade: ●●●

```
class Relatorio {  
    private BancoDeDados db;  
  
    public List<Venda> buscarVendas() {  
        return db.query("SELECT * FROM vendas");  
    }  
}
```

Introduzir uma interface VendaRepository entre Relatorio e BancoDeDados representa qual padrão GRASP?

- A) Pure Fabrication, pois VendaRepository é uma classe inventada sem correspondência no domínio.
- B) Indirection, pois introduz um intermediário que desacopla Relatorio do mecanismo concreto de persistência.
- C) Controller, pois VendaRepository passa a coordenar o acesso aos dados.
- D) Expert, pois VendaRepository terá o conhecimento especializado sobre como acessar vendas.
- E) As opções A e B estão ambas corretas para cenários distintos.

## Questão 9 Protected Variations Dificuldade: ●●●

```
class ProcessadorPagamento {
    public void processar(Cartao cartao, double valor) {
        if (cartao.getBandeira().equals("VISA")) {
            VisaGateway.cobrar(cartao.getNumero(), valor);
        } else {
            MastercardGateway.cobrar(cartao.getNumero(), valor);
        }
    }
}
```

Qual padrão GRASP orienta a solução correta para proteger o sistema quando uma nova bandeira de cartão for adicionada?

- A) High Cohesion: mover cada if para sua própria classe para reduzir o tamanho do método.
- B) Low Coupling: remover as dependências de VisaGateway e MastercardGateway da classe.
- C) Protected Variations: introduzir uma interface GatewayPagamento que isole as variações de comportamento por bandeira.
- D) Creator: atribuir a responsabilidade de criar o gateway correto ao próprio Cartao.
- E) Controller: criar um GatewayController que roteie para o gateway correto com base na bandeira.

## Questão 10 Controller vs. Expert Dificuldade: ●●●

```
class CarrinhoController {
    private CarrinhoService service;

    public void adicionarItem(String sessao, int produtoId, int qtd) {
        Carrinho c = service.buscarPorSessao(sessao);
        c.adicionar(produtoId, qtd); // logica no dominio
        service.salvar(c);
    }
}
```

Comparando os padrões Controller e Expert neste design, qual afirmativa é correta?

- A) O Controller viola o Expert porque não calcula diretamente o novo estado do carrinho.
- B) O design aplica corretamente ambos os padrões: Controller coordena o fluxo e Carrinho (Expert) executa a lógica de negócio de adicionar itens.
- C) O design viola o Controller porque CarrinhoController chama c.adicionar() diretamente em vez de delegar ao service.
- D) O design viola o Low Coupling porque CarrinhoController depende tanto de CarrinhoService quanto de Carrinho.
- E) O design viola o Creator porque CarrinhoController não cria o Carrinho.

## Gabarito e Explicações

### Questão 1 — Resposta: A [Expert (Information Expert)]

Expert determina que a responsabilidade deve ficar com quem tem a informação. FaturaService não possui linhas nem desconto; para calcular, precisaria receber ou acessar esses dados, violando o Expert. A opção D menciona um efeito colateral real, mas não é a violação primária — é consequência, não causa.

### Questão 2 — Resposta: B [Creator]

O GRASP Creator não é uma regra absoluta — é uma heurística. Quando a criação envolve lógica complexa ou precisa variar, delegar a uma fábrica é justificado. A opção C erra ao tratar o Creator como regra inviolável. A opção A ignora que GoF e GRASP atuam em níveis diferentes.

### Questão 3 — Resposta: C [High Cohesion]

enviarEmailBoas\_vindas pertence ao domínio de notificação/infraestrutura; calcularDescontoFidelidade ao domínio de precificação/marketing. Ambos são extrínsecos à identidade do usuário. Extraí-los para ServicoNotificacao e ServicoPrecificacao resulta no maior ganho de coesão.

### Questão 4 — Resposta: A [Low Coupling]

Isso é a Lei de Deméter aplicada via Low Coupling. Relatorio acessa objetos que não são parâmetros diretos seus — depende de Cliente e Status transitivamente. Se a estrutura interna de Pedido mudar, Relatorio quebra. A solução GRASP seria mover a lógica de formatação para Pedido (Expert) ou criar um DTO.

### Questão 5 — Resposta: B [Controller]

O GRASP Controller deve receber e coordenar requisições, delegando lógica às camadas inferiores. Aqui, o Controller implementa a regra de compensação (liberar estoque se pagamento falhar) — lógica de negócio que deveria estar num serviço de aplicação ou saga. A opção D é superficial — ter três dependências não viola Low Coupling por si só.

### Questão 6 — Resposta: B [Polymorphism]

O GRASP Polimorfismo afirma: quando o comportamento varia por tipo, use polimorfismo em vez de condicionais. Cada canal vira uma implementação de uma interface Notificador, eliminando o if-else e tornando o sistema aberto a extensão (Open/Closed). A injeção de dependência (D) é complementar, não a solução estrutural.

### Questão 7 — Resposta: B [Pure Fabrication]

Pure Fabrication é exatamente isso: uma classe que não existe no domínio do problema, criada para separar responsabilidades técnicas sem poluir as entidades de domínio. Repository não é um conceito do negócio — é uma invenção de design. As opções D e E descrevem consequências corretas da separação, mas não nomeiam o padrão.

### Questão 8 — Resposta: E [Indirection]

Pure Fabrication e Indirection são dois padrões GRASP distintos mas frequentemente aplicados juntos. VendaRepository é uma Pure Fabrication (classe inventada, sem existência no domínio). Ao mesmo tempo, serve como Indirection (intermediário que desacopla Relatorio do banco concreto). A resposta E reconhece essa sobreposição — os dois padrões coexistem.

### Questão 9 — Resposta: C [Protected Variations]

Protected Variations diz: identifique os pontos de variação e crie uma interface estável ao redor deles. Adicionar uma nova bandeira hoje exige modificar ProcessadorPagamento. Criando interface GatewayPagamento, o ponto de variação fica encapsulado. Low Coupling (B) é um benefício resultante, não o padrão que nomeia a intenção.

---

**Questão 10 — Resposta: B** [Controller vs. Expert]

Este é o design ideal segundo GRASP. Controller: CarrinhoController recebe a requisição e coordena os passos (buscar, delegar, salvar) sem implementar lógica de domínio. Expert: Carrinho possui seus próprios dados e implementa a regra de adicionar itens. A separação entre coordenação e comportamento de domínio é exatamente o que esses dois padrões pregam em conjunto.